山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级： 人工智能班
实验题目：实验 1 递归练习		
实验学时：2	实验日期： 2023.9.13	

实验目的：

（1）熟悉开发工具的使用。

（2）掌握递归的实现思想

软件开发工具：

Dev c++

1. 实验内容

（1）、 键盘输入 n 和 n 个互不相同的整数，输出 n 个整数的全排列。

（2）、 键盘输入 n 和 n 个互不相同的整数，输出 n 个整数的所有子集。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

A：子集价值

问题分析：最终要求得所有子集价值的异或和，首先要求得所有子集的子集价值。第一步要找到所有的子集。集合的子集有 2^n 个，容易联想到 n 位 2 进制数能够表达 2^n 个状态。所以要求得所有的子集，可以考虑成集合中的元素是否存在新的集合中，并由此递归生成一颗树，求得所有子集。再将子集中的元素与序号相乘得到子集价值。子集价值之间再进行异或预算得到最终结果。

数据结构与算法：递归，模拟二进制数、树

B：全排列问题

问题分析：该问题要将得到的全排列的序号与元素异或并加和，再将所有的加和结果进行或运算。所以首先要考虑如何求得所有的全排列。可以通过确定 n 位元素排列中第一个元素，再递归求解有 n-1 位元素排列的全排列，从而求出所有的全排列。求出全排列后，每个排列进行序号与元素的异或并求和，再将所有的和进行或操作求出最终答案。

数据结构与算法：递归

3. 测试结果（测试输入，测试输出）

A：子集价值

输入：2 1 2

输出：6

B：全排列问题

输入：3 1 2 3

输出：6

4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

A，B 的结果正确无误，思路正确。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

A：子集价值代码

```
#include<iostream>

class Solution{
public:
    Solution(int k,int n):k(k),n(n){
    };
    void deal();
    void solve(int k, int n);
    void getAdata();
    void getBdata();
    void OutAns();
    ~Solution(){
        delete []a;
        delete []b;
    }
private:
    int ans = 0;
    int sum = 0;
    int n;
    int k;
    int *a = new int [30];   //a 数组用于盛放给出的序列
    int *b = new int [30];   //b 数组初始为 0，暂时用来盛放被交换的 a 数组中元素
};
void Solution::solve(int k, int n){
    if(k==n){             // 当进行到第 n 个元素时，由于在输入时 n 比序列数大 1，
所以已经完成了对所有序列数的处理操作
        sum = 0;
        int count = 0;        //因为有的子集中有多个 0 元素，实际上这些 0 代表没有
序列数，记录个数用以循环时跳过这些 0
        for(int i = 1; i <= n; i++){
            if(a[i]==0){
                count++;   //如果这个元素是 0，说明该位置没有元素，count++
            }
            if(a[i]!=0){
                sum+=a[i]*(i-count); //计算序列的 序号与元素之积的和
            }
        }
```

```
        ans = ans^sum;
        return ;
    }
    solve(k+1,n);         //首先认为这个元素在生成的子集中，所以直接考虑下一个
元素
    std::swap(a[k],b[k]);    //将本元素与 b 数组中 0 元素交换，认为
    solve(k+1,n);         //认为这个元素不在生成的子集中，再考虑下一个元素
    std::swap(a[k],b[k]);    //将被交换的两个元素归位
}
void Solution::getAdata(){
    for(int i = 1; i <= n; i++){
    scanf("%d",&a[i]);
    }
    a[n+1] = 0;
}
void Solution::getBdata(){
    for(int i = 1; i <= n; i++){
        b[i] = 0;
    }
}
void Solution::OutAns(){
    std::cout<<ans;
}
void Solution::deal(){
    solve(k, n + 1);      //传入的数据是 n + 1,比序列元素数多一个
}

int main(){
    int n;scanf("%d",&n);
    Solution s(1,n);
    s.getAdata();
    s.getBdata();
    s.deal();
    s.OutAns();
    return 0;
    }
```

B:全排列问题代码

```
#include<iostream>


class Solution{
public:
    Solution(int k, int n):k(k),n(n){};
    void deal();              //暴露给用户的问题处理函数
    void solve(int k, int n);   //被隐藏封装的问题处理函数
```

```
    void getAdata();          //接收数组 a 的数据
    void OutAns();
    ~Solution(){
        delete []a;
    }
private:
    int ans = 0;              //用来保存最终答案
    int sum = 0;              //用来保存每一次排列的序号与元素的异或和
    int *a = new int [30];
    int n;
    int k;
};

void Solution::solve(int k,int n){
    if(k==n){                 //当操作数序号等于 n 时，说明已经是最后一个元素，此
时完成一次排列
        sum = 0;
        for(int i = 1; i <= n; i++){
            sum += i^a[i];        //计算此次排列的异或和
        }
        ans= ans|sum;         //此次的异或和与要输出的答案或运算
}
    for(int i = k;i <= n; i++){ //将该位置元素与元素位置以后的每个元素依次交
换，
        std::swap(a[k], a[i]);  //交换该位置元素与第 i 个位置元素
        solve(k+1, n);          //确定该位置元素后，递归求解剩余元素的全排列问
题
        std::swap(a[k], a[i]); //将交换过的元素归位，进行下一个元素的交换
    }
}
void Solution::getAdata(){      //输入序列
    for(int i = 1; i <= n; i++){
        scanf("%d",&a[i]);
    }
}
void Solution::deal(){
    solve(k, n);
}
void Solution::OutAns(){
    std::cout<<ans;             //输出答案
}
int main(){
    int n;scanf("%d",&n);
    Solution s(1, n);
    s.getAdata();
    s.deal();
```

```
    s.OutAns();
    return 0;
}
```

6.